

HW2_114062545

Docker

Task 1: Building the images

1. Enter the client folder, open and modify the **PLEASE ASSIGN** section and append your student ID in **socket_client.c** as shown in the following image:

修改 socket_client.c 檔案

Steps:

1. 修改 SERVER_ADDRESS

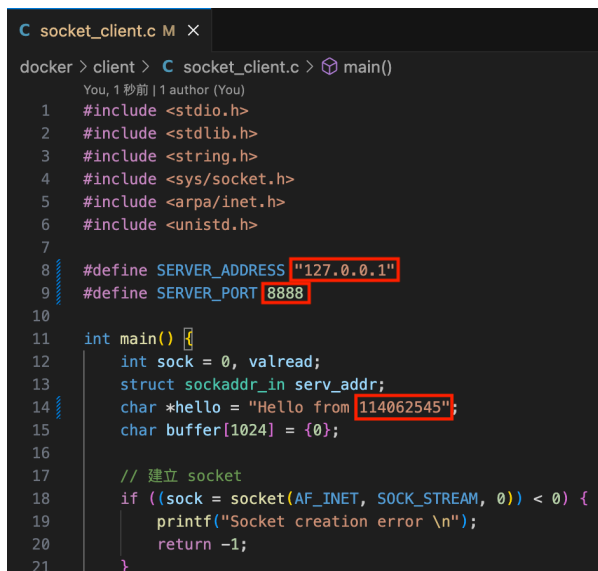
- 將 `#define SERVER_ADDRESS "PLEASE ASSIGN"` 改為 `#define SERVER_ADDRESS "127.0.0.1"`
- 原因：127.0.0.1 是本地主機 (localhost) 的 IP 位址，用於在本機上進行客戶端與伺服器之間的通訊測試

2. 修改 SERVER_PORT

- 將 `#define SERVER_PORT PLEASE ASSIGN` 改為 `#define SERVER_PORT 8888`
- 原因：8888 是伺服器監聽的 port，客戶端需要連接到相同的 port 才能與伺服器建立連線

3. 加入學號到 Hello 訊息

- 將 `char *hello = "Hello from <your student ID>";` 改為 `char *hello = "Hello from 114062545";`
- 原因：根據要求，需要在訊息中加入學號 (114062545)，以便識別是誰發送的訊息



```
C socket_client.c M x
docker > client > C socket_client.c > main()
You, 1 秒前 | 1 author (You)
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <sys/socket.h>
5 #include <arpa/inet.h>
6 #include <unistd.h>
7
8 #define SERVER_ADDRESS "127.0.0.1"
9 #define SERVER_PORT 8888
10
11 int main() {
12     int sock = 0, valread;
13     struct sockaddr_in serv_addr;
14     char *hello = "Hello from 114062545";
15     char buffer[1024] = {0};
16
17     // 建立 socket
18     if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
19         printf("Socket creation error \n");
20         return -1;
21     }
```

2. Build the image with the Dockerfile

在執行 docker build 之前，需確認當前使用者具有 Docker 執行權限，可使用指令 `id` 檢查是否在 docker group 中，如下圖所示：

```
~/.virtlab virtlab-hw2 !2
> id
uid=1045(DannyTso) gid=1045(DannyTso) groups=1045(DannyTso),27(sudo),100(users),998(docker)
```

若不在 docker group 中，可選擇：

- 將使用者加入 docker group：`sudo usermod -aG docker DannyTso`
- 或在每次執行指令前加上 `sudo`

確認權限後，再切換到當前使用者 `su - DannyTso`，執行 build 指令：

```
docker build -t server-114062545 .
docker build -t client-114062545 .
```

再來，check images we've built：

docker images

```
~/.virtlab/docker/server virtlab-hw2 !1
docker build -t server-114062545 .
[*] Building 0.4s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring Dockerfile: 200B
=> [internal] load metadata for docker.io/library/gcc:12.2.0
=> [internal] load .dockerignore
=> transferring context: 26
=> [internal] load build context
=> transferring context: 17.04kB
=> CACHED [1/4] FROM docker.io/library/gcc:12.2.0@sha256:a3e091325cda43bc9c1c576ddd155351d5b16438124
=> [2/4] COPY socket_server /src/server
=> CACHED [3/4] RUN chmod +x /src/server
=> [4/4] WORKDIR /src
=> exporting to image
=> exporting layers
=> writing image sha256:b7c24898a146e175f04ad6280cf3fc2c37b5afa54e04e0e22004ea2c16f
=> naming to docker.io/library/server-114062545

~/.virtlab/docker/client virtlab-hw2 !1
docker build -t client-114062545 .
[*] Building 1.5s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring Dockerfile: 200B
=> [internal] load metadata for docker.io/library/gcc:12.2.0
=> [internal] load .dockerignore
=> transferring context: 26
=> [internal] load build context
=> transferring context: 37B
=> [1/4] FROM docker.io/library/gcc:12.2.0@sha256:a3e091325cda43bc9c1c576ddd155351d5b16438124421188b
=> CACHED [2/4] COPY socket_client.c /src/socket_client.c
=> CACHED [3/4] WORKDIR /src
=> CACHED [4/4] RUN gcc -o client socket_client.c
=> exporting to image
=> exporting layers
=> writing image sha256:022aef92775c3bb36c113bd615bb02235050e92b24dc4c1c56422b7478689e1
=> naming to docker.io/library/client-114062545

~/.virtlab/docker/server virtlab-hw2 !1
> docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
server-114062545    latest             b7c24898a14        14 seconds ago    1.27GB
client-114062545    latest             022aef92775c       20 minutes ago    1.27GB
server-113062566    latest             efed5cb43b76       3 weeks ago       1.27GB
client-113062566    latest             2ac3e310dc96       3 weeks ago       1.27GB
gcr.io/k8s-minikube/kicbase v0.0.48            c905532e9b7b       6 weeks ago       1.31GB
dasdb72/xv6         amd64              7885caff6127       4 months ago     1.4GB
ghcr.io/nthu-cp-01/final main               fd5bee93622a       6 months ago     1.11GB
cpr.dev/chainguard/wait-for-it latest             e819634f3a9b       2 years ago       12.4MB
nginx               <none>             61395b4c586d       2 years ago       187MB
istio/proxyv2       1.19.0            5df444b77d96       2 years ago       253MB
istio/pilot         1.19.0            77dc344ac7c7       2 years ago       179MB
alpine              latest             7e01a0b0a1dc       2 years ago       7.34MB
kindest/node        <none>             89e7dc9f9131       2 years ago       932MB
registry.k8s.io/kube-proxy v1.27.1           fae39e5d6d06       2 years ago       71.1MB
registry.k8s.io/coredns/coredns v1.10.1           ead0a4a53d98       2 years ago       53.6MB
calico/kube-controllers v3.25.0           5e785d005ccc       2 years ago       71.6MB
calico/cni           v3.25.0           d70a5947d57e       2 years ago       189MB
calico/node          v3.25.0           0861626b8e67       2 years ago       249MB
ghcr.io/spiffe/spire-agent 1.5.1            7c6f86deab98       2 years ago       36MB
registry.k8s.io/pause 3.6              6278b6b60e12       4 years ago       683kB

~/.virtlab/docker/client virtlab-hw2 !1
> docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
server-114062545    latest             b7c24898a14        20 seconds ago    1.27GB
client-114062545    latest             022aef92775c       20 minutes ago    1.27GB
server-113062566    latest             efed5cb43b76       3 weeks ago       1.27GB
client-113062566    latest             2ac3e310dc96       3 weeks ago       1.27GB
gcr.io/k8s-minikube/kicbase v0.0.48            c905532e9b7b       6 weeks ago       1.31GB
dasdb72/xv6         amd64              7885caff6127       4 months ago     1.4GB
ghcr.io/nthu-cp-01/final main               fd5bee93622a       6 months ago     1.11GB
cpr.dev/chainguard/wait-for-it latest             e819634f3a9b       2 years ago       12.4MB
nginx               <none>             61395b4c586d       2 years ago       187MB
istio/proxyv2       1.19.0            5df444b77d96       2 years ago       253MB
istio/pilot         1.19.0            77dc344ac7c7       2 years ago       179MB
alpine              latest             7e01a0b0a1dc       2 years ago       7.34MB
kindest/node        <none>             89e7dc9f9131       2 years ago       932MB
registry.k8s.io/kube-proxy v1.27.1           fae39e5d6d06       2 years ago       71.1MB
registry.k8s.io/coredns/coredns v1.10.1           ead0a4a53d98       2 years ago       53.6MB
calico/kube-controllers v3.25.0           5e785d005ccc       2 years ago       71.6MB
calico/cni           v3.25.0           d70a5947d57e       2 years ago       189MB
calico/node          v3.25.0           0861626b8e67       2 years ago       249MB
ghcr.io/spiffe/spire-agent 1.5.1            7c6f86deab98       2 years ago       36MB
registry.k8s.io/pause 3.6              6278b6b60e12       4 years ago       683kB
```

3. Launch the server container first, and then launch the client container:

```
docker run -it --rm --network host \
--name server \
server-114062545
docker run -it --rm --network host \
--name client \
client-114062545
```

```

[1] ~ /virtlab/docker/server virtlab-hw2 11 ..... DannyTsoq@k8snode2 18:30:41
> docker run -it --rm --network host \
  --name server \
  server-114062545
Server started listening on port 8888
Received message from Client: Hello from 114062545
Response sent to Client

[2] ~ /virtlab/docker/server virtlab-hw2 11 ..... DannyTsoq@k8snode2 18:32:42
[2] @tssh-

[1] ~ /virtlab/docker/client virtlab-hw2 11 ..... DannyTsoq@k8snode2 18:30:47
> docker run -it --rm --network host \
  --name client \
  client-114062545
Hello message sent.
Hello from Server

[2] ~ /virtlab/docker/client virtlab-hw2 11 ..... DannyTsoq@k8snode2 18:32:42
[2] @tssh-
"K8snode2" 18:33 05-11-25

```

Task 2: Creating bridge

1. Create 2 containers using the following commands:

```
docker container run -d --rm --name box1 busybox /bin/sh -c "while true; do sleep 3600; done"
```

```
docker container run -d --rm --name box2 busybox /bin/sh -c "while true; do sleep 3600; done"
```

```

[1] ~ /virtlab/docker/server virtlab-hw2 11 ..... DannyTsoq@k8snode2 19:09:30
> docker container run -d --rm --name box1 busybox /bin/sh -c "while true; do sleep 3600; done"
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
e59838ecfec5: Pull complete
Digest: sha256:e3652a0a2fabd16ce889f0aa32c38eec347b997e73bd09e69c962ec7f8732ee
Status: Downloaded newer image for busybox:latest
db146cc7727e02d2a196bd12f9d162adb4f6e58b3cab403c99120c6ee0c1c8e

[2] ~ /virtlab/docker/client virtlab-hw2 11 ..... DannyTsoq@k8snode2 19:09:34
> docker container run -d --rm --name box2 busybox /bin/sh -c "while true; do sleep 3600; done"
4fa56215b1013fca2b0e1b883a752e5f8432a2533e562c5d85d6ecd179e5a97

[2] ~ /virtlab/docker/client virtlab-hw2 11 ..... DannyTsoq@k8snode2 19:09:53
>
"K8snode2" 19:10 05-11-25

```

2. Create a network bridge:

```
docker network create box-bridge
```

```

[1] ~ /virtlab/docker/server virtlab-hw2 11 ..... DannyTsoq@k8snode2 19:09:30
> docker container run -d --rm --name box1 busybox /bin/sh -c "while true; do sleep 3600; done"
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
e59838ecfec5: Pull complete
Digest: sha256:e3652a0a2fabd16ce889f0aa32c38eec347b997e73bd09e69c962ec7f8732ee
Status: Downloaded newer image for busybox:latest
db146cc7727e02d2a196bd12f9d162adb4f6e58b3cab403c99120c6ee0c1c8e

[2] ~ /virtlab/docker/client virtlab-hw2 11 ..... DannyTsoq@k8snode2 19:09:34
> docker container run -d --rm --name box2 busybox /bin/sh -c "while true; do sleep 3600; done"
4fa56215b1013fca2b0e1b883a752e5f8432a2533e562c5d85d6ecd179e5a97

[2] ~ /virtlab/docker/client virtlab-hw2 11 ..... DannyTsoq@k8snode2 19:09:53
> docker network create box-bridge
ab8957a1b0c141a3102ea21628386745587a2219df53cd32c947a47036566869

[2] ~ /virtlab/docker/client virtlab-hw2 11 ..... DannyTsoq@k8snode2 19:11:33
>
"K8snode2" 19:11 05-11-25

```

3. Connecting two running containers to the network bridge:

```
docker network connect box-bridge box1
```

```
docker network connect box-bridge box2
```

```

[1] ~ /virtlab/docker/server virtlab-hw2 11 ..... DannyTsoq@k8snode2 19:09:30
> docker container run -d --rm --name box1 busybox /bin/sh -c "while true; do sleep 3600; done"
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
e59838ecfec5: Pull complete
Digest: sha256:e3652a0a2fabd16ce889f0aa32c38eec347b997e73bd09e69c962ec7f8732ee
Status: Downloaded newer image for busybox:latest
db146cc7727e02d2a196bd12f9d162adb4f6e58b3cab403c99120c6ee0c1c8e

[2] ~ /virtlab/docker/client virtlab-hw2 11 ..... DannyTsoq@k8snode2 19:09:34
> docker container run -d --rm --name box2 busybox /bin/sh -c "while true; do sleep 3600; done"
4fa56215b1013fca2b0e1b883a752e5f8432a2533e562c5d85d6ecd179e5a97

[2] ~ /virtlab/docker/client virtlab-hw2 11 ..... DannyTsoq@k8snode2 19:09:53
> docker network create box-bridge
ab8957a1b0c141a3102ea21628386745587a2219df53cd32c947a47036566869

[2] ~ /virtlab/docker/client virtlab-hw2 11 ..... DannyTsoq@k8snode2 19:11:33
> docker network connect box-bridge box1

[2] ~ /virtlab/docker/client virtlab-hw2 11 ..... DannyTsoq@k8snode2 19:14:10
>
"K8snode2" 19:14 05-11-25

```

4. You can use the following command to list docker networks:

docker network ls

```
19:09:30 > docker container run -d --name box1 busybox /bin/sh -c "while true; do sleep 3600; done"
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
c59830cfec5f: Pull complete
Digest: sha256:e3652a0a27abd16ce889f0aa32c38ec347b997e73bd99e9c962ec7f8732ee
Status: Downloaded newer image for busybox:latest
d0146cc7727e022a196bd12f9d162adb4f6e58b3caba403c99120c6ee8c1c8e

19:09:34 > docker container run -d --name box2 busybox /bin/sh -c "while true; do sleep 3600; done"
4fa56215b1013fca20e1b883a752e5f8432a2553e562c5d85d6ecd17f9e5a97

19:09:53 > docker network create box-bridge
a8d957a1b0c141a3102ea21620306745507a2219df53cd32c947a47036566869

19:11:33 > docker network connect box-bridge box1

19:14:10 > docker network connect box-bridge box2

19:15:37 > docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
a8d957a1b0c1        box-bridge          bridge              local
64380f695f89        bridge              bridge              local
a8a0e361129a        host                host                local
d06e69318397        kind                bridge              local
a788b487dcfe        none                null                local
```

5. You can inspect more about the details of the **box-bridge** network:

docker network inspect box-bridge

```
19:13:55 > docker network inspect box-bridge
[{"Name": "box-bridge",
  "Id": "a8d957a1b0c141a3102ea21620306745507a2219df53cd32c947a47036566869",
  "Created": "2025-11-05T19:11:33.545439522+08:00",
  "Scope": "local",
  "Driver": "bridge",
  "EnableIPv4": true,
  "EnableIPv6": false,
  "IPAM": {
    "Driver": "default",
    "Options": {},
    "Config": [
      {
        "Subnet": "172.19.0.0/16",
        "Gateway": "172.19.0.1"
      }
    ]
  },
  "Internal": false,
  "Attachable": false,
  "Ingress": false,
  "ConfigFrom": {
    "Network": ""
  },
  "ConfigOnly": false,
  "Containers": {
    "4fa56215b1013fca20e1b883a752e5f8432a2553e562c5d85d6ecd17f9e5a97": {
      "Name": "box2",
      "EndpointID": "cd12f8f5978e88f734464ea21e4d4bd14ae8c48e9552ee7efc99ee9143f0ba5",
      "MacAddress": "86:7e:35:104:b4:57",
      "IPv4Address": "172.19.0.3/16",
      "IPv6Address": ""
    },
    "d0146cc7727e022a196bd12f9d162adb4f6e58b3caba403c99120c6ee8c1c8e": {
      "Name": "box1",
      "EndpointID": "91df941ab2840138bb7217b4c773571ac9cf5401d56c74e4f8d4271ba360096",
      "MacAddress": "ba:c2:b5:48:a3:ef",
      "IPv4Address": "172.19.0.2/16",
      "IPv6Address": ""
    }
  },
  "Options": {},
  "Labels": {}
}]
```

6. Log into **box1** and try to ping **box2** :

```
docker exec -it box1 sh
ping box2
```

p.s. **Ctrl + c** : abort 掉剛剛 ping 這個動作

```
問題 輸出 擴展主控台 終端機 選擇器 GITLENS + v tmux - client 19:09:38
> docker container run -d --rm --name box1 busybox /bin/sh -c "while true; do sleep 3600; done"
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
e59838ecfec3: Pull complete
Digest: sha256:e3652a0ba2fabd16ce889f8aa32c38ec347b997673bd09e6c962ec7f8732ee
Status: Downloaded newer image for busybox:latest
db146cc7727e02a196bd12f9d162adb4f6e58b3cab403c99120c6ee8c18e

19:13:55
> docker network connect box-bridge box1

19:13:55
> ping box2
PING box2 (172.19.0.3): 56 data bytes
64 bytes from 172.19.0.3: seq=0 ttl=64 time=0.077 ms
64 bytes from 172.19.0.3: seq=1 ttl=64 time=0.092 ms
64 bytes from 172.19.0.3: seq=2 ttl=64 time=0.074 ms
64 bytes from 172.19.0.3: seq=3 ttl=64 time=0.078 ms
64 bytes from 172.19.0.3: seq=4 ttl=64 time=0.085 ms
64 bytes from 172.19.0.3: seq=5 ttl=64 time=0.064 ms
64 bytes from 172.19.0.3: seq=6 ttl=64 time=0.088 ms
64 bytes from 172.19.0.3: seq=7 ttl=64 time=0.071 ms
64 bytes from 172.19.0.3: seq=8 ttl=64 time=0.068 ms
64 bytes from 172.19.0.3: seq=9 ttl=64 time=0.081 ms
64 bytes from 172.19.0.3: seq=10 ttl=64 time=0.067 ms
64 bytes from 172.19.0.3: seq=11 ttl=64 time=0.080 ms
64 bytes from 172.19.0.3: seq=12 ttl=64 time=0.100 ms
!

19:17:13
> docker network inspect box-bridge
{
  "Name": "box-bridge",
  "IP": "ab8957a1b8c1a1a180ea21628386745507a2219ef53cd32c947a47036566869",
  "Created": "2025-11-05T19:11:33.345439522+08:00",
  "Scope": "local",
  "Driver": "bridge",
  "EnableIPv6": true,
  "EnableIPv6": false,
  "IPAM": {
    "Driver": "default",
    "Options": {},
    "Config": [
      {
        "Subnet": "172.19.0.0/16",
        "Gateway": "172.19.0.1"
      }
    ]
  },
  "Internal": false,
  "Attachable": false,
  "Ingress": false,
  "ConfigFrom": {
    "Network": ""
  },
  "ConfigOnly": false,
  "Containers": {
    "4fa5621b1013fca020e1b883a752e5f8432a2553e562c5d85decd17f9e5a97": {
      "Name": "box2",
      "EndpointID": "cd12f9f5978e8f734464ea21e4d4bd14ae8c48e9552ee72efc99e9143f80a5",
      "MacAddress": "d6:7e:35:84:b4:57",
      "IPv4Address": "172.19.0.3/16",
      "IPv6Address": ""
    },
    "db146cc7727e02a196bd12f9d162adb4f6e58b3cab403c99120c6ee8c18e": {
      "Name": "box1",
      "EndpointID": "91df91ab284813bb7217b4c773571ac9cf5401d56c74e4f8d4271ba368096",
      "MacAddress": "ba:c2:1b5:48:a3:ef",
      "IPv4Address": "172.19.0.2/16",
      "IPv6Address": ""
    }
  },
  "Options": {},
  "Labels": {}
}
```

```
問題 輸出 擴展主控台 終端機 選擇器 GITLENS + v tmux - client 19:09:38
> docker container run -d --rm --name box1 busybox /bin/sh -c "while true; do sleep 3600; done"
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
e59838ecfec3: Pull complete
Digest: sha256:e3652a0ba2fabd16ce889f8aa32c38ec347b997673bd09e6c962ec7f8732ee
Status: Downloaded newer image for busybox:latest
db146cc7727e02a196bd12f9d162adb4f6e58b3cab403c99120c6ee8c18e

19:13:55
> docker network connect box-bridge box1

19:13:55
> ping box2
PING box2 (172.19.0.3): 56 data bytes
64 bytes from 172.19.0.3: seq=0 ttl=64 time=0.077 ms
64 bytes from 172.19.0.3: seq=1 ttl=64 time=0.092 ms
64 bytes from 172.19.0.3: seq=2 ttl=64 time=0.074 ms
64 bytes from 172.19.0.3: seq=3 ttl=64 time=0.078 ms
64 bytes from 172.19.0.3: seq=4 ttl=64 time=0.085 ms
64 bytes from 172.19.0.3: seq=5 ttl=64 time=0.064 ms
64 bytes from 172.19.0.3: seq=6 ttl=64 time=0.088 ms
64 bytes from 172.19.0.3: seq=7 ttl=64 time=0.071 ms
64 bytes from 172.19.0.3: seq=8 ttl=64 time=0.068 ms
64 bytes from 172.19.0.3: seq=9 ttl=64 time=0.081 ms
64 bytes from 172.19.0.3: seq=10 ttl=64 time=0.067 ms
64 bytes from 172.19.0.3: seq=11 ttl=64 time=0.080 ms
64 bytes from 172.19.0.3: seq=12 ttl=64 time=0.084 ms
64 bytes from 172.19.0.3: seq=13 ttl=64 time=0.085 ms
64 bytes from 172.19.0.3: seq=14 ttl=64 time=0.081 ms
64 bytes from 172.19.0.3: seq=15 ttl=64 time=0.064 ms
64 bytes from 172.19.0.3: seq=16 ttl=64 time=0.079 ms
64 bytes from 172.19.0.3: seq=17 ttl=64 time=0.070 ms
64 bytes from 172.19.0.3: seq=18 ttl=64 time=0.081 ms
^C
-- box1 ping statistics --
19 packets transmitted, 19 packets received, 0% packet loss
round-trip min/avg/max = 0.064/0.078/0.100 ms
/ # !

19:17:13
> docker network inspect box-bridge
{
  "Name": "box-bridge",
  "IP": "ab8957a1b8c1a1a180ea21628386745507a2219ef53cd32c947a47036566869",
  "Created": "2025-11-05T19:11:33.345439522+08:00",
  "Scope": "local",
  "Driver": "bridge",
  "EnableIPv6": true,
  "EnableIPv6": false,
  "IPAM": {
    "Driver": "default",
    "Options": {},
    "Config": [
      {
        "Subnet": "172.19.0.0/16",
        "Gateway": "172.19.0.1"
      }
    ]
  },
  "Internal": false,
  "Attachable": false,
  "Ingress": false,
  "ConfigFrom": {
    "Network": ""
  },
  "ConfigOnly": false,
  "Containers": {
    "4fa5621b1013fca020e1b883a752e5f8432a2553e562c5d85decd17f9e5a97": {
      "Name": "box2",
      "EndpointID": "cd12f9f5978e8f734464ea21e4d4bd14ae8c48e9552ee72efc99e9143f80a5",
      "MacAddress": "d6:7e:35:84:b4:57",
      "IPv4Address": "172.19.0.3/16",
      "IPv6Address": ""
    },
    "db146cc7727e02a196bd12f9d162adb4f6e58b3cab403c99120c6ee8c18e": {
      "Name": "box1",
      "EndpointID": "91df91ab284813bb7217b4c773571ac9cf5401d56c74e4f8d4271ba368096",
      "MacAddress": "ba:c2:1b5:48:a3:ef",
      "IPv4Address": "172.19.0.2/16",
      "IPv6Address": ""
    }
  },
  "Options": {},
  "Labels": {}
}
```

Task 3: Creating volume

Part 0: 新開一個 terminal shell 並做好前置作業

```
問題 輸出 擴展主控台 終端機 選擇器 GITLENS + v tmux - client 19:34:53
> su - DannyTso
Password:

19:35:07
> id
uid=1045(DannyTso) gid=1045(DannyTso) groups=1045(DannyTso),27(sudo),100(users),998(docker)

19:35:10
>
```

Part 1: You need to use `docker volume create` to create a volume and share the volume between two containers

p.s. 先用 `docker volume ls` check 已建立的 volume

```
~/virtlab virtlab-hw2 !2
> docker volume ls
DRIVER      VOLUME NAME
local       80aa686723ad3f1f0c662cd8120c8e45a8cddb77961bbcc38364d1ebb8ea8db9
local       minikube
```

1. 建立一個 volume

```
docker volume create my-shared-volume
```

2. 啟動第一個容器，掛載 volume

```
docker run -dit --name container1 \
-v my-shared-volume:/data \
busybox
```

3. 在 container1 中建立檔案 (切記：要逐行執行)

```
docker exec -it container1 sh
cd /data
echo "Hello from container1" > test.txt
ls -la
exit
```

4. 啟動第二個容器，掛載同一個 volume

```
docker run -dit --name container2 \
-v my-shared-volume:/data \
busybox
```

5. 在 container2 中讀取檔案 (切記：要逐行執行)

```
docker exec -it container2 sh
cd /data
cat test.txt
echo "Reply from container2" >> test.txt
cat test.txt
exit
```

6. 驗證資料已同步

```
docker exec container1 cat /data/test.txt
```

7. 檢查 volume 資訊

```
docker volume inspect my-shared-volume
docker volume ls
```

Step1~6

```
❏ [19:52:00 (21/22)]
> docker volume create my-shared-volume
my-shared-volume

❏ [19:48:16]
> docker run -dit --name container1 \
  -v my-shared-volume:/data \
  busybox
085b62dec8ff08d14c268ea951a82d5c86fc4a4627fc33c59582aecb0853d85

❏ [19:48:25]
> docker exec -it container1 sh
/# cd /data
/data # echo "Hello from container1" > test.txt
/data # ls -la
total 12
drwxr-xr-x 2 root root 4096 Nov 5 11:48 .
drwxr-xr-x 1 root root 4096 Nov 5 11:48 ..
-rw-r--r-- 1 root root 22 Nov 5 11:48 test.txt
/data # exit

❏ [26s 19:49:15]
> docker container ls
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
085b62dec8ff  busybox    "sh"                    About a minute ago    Up About a minute    127.0.0.1:43217->6443/tcp    container1
4fa56215b101  busybox    "/bin/sh -c 'while t..." 39 minutes ago    Up 39 minutes                               box2
db146cc7727e  busybox    "/bin/sh -c 'while t..." 39 minutes ago    Up 39 minutes                               box1
abc1d69d3d99  kindest/node:v1.27.3  "/usr/local/bin/entr..." 2 years ago      Up 10 days                               kind-control-plane

❏ [19:49:26]
> docker exec container1 cat /data/test.txt
Hello from container1

❏ [19:49:36]
> docker run -dit --name container2 \
  -v my-shared-volume:/data \
  busybox
8980e20099db4168dde5eb5b59dac98bca7686d5afdf86a952e45127ad2e8

❏ [19:49:48]
> docker exec -it container2 sh
/# cd /data
/data # cat test.txt
Hello from container1
/data # echo "Reply from container2" >> test.txt
/data # cat test.txt
Hello from container1
Reply from container2
/data # exit

❏ [27s 19:51:05]
> docker exec container1 cat /data/test.txt
Hello from container1
Reply from container2

❏ [19:51:11]
> docker volume inspect my-shared-volume
[2] 01zsh 11zsh-211zsh+ "k8snode2" 19:54 05-11-25
```

Step7

```
❏ [19:51:11]
> docker volume inspect my-shared-volume
[
  {
    "CreatedAt": "2025-11-05T19:48:16+08:00",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/my-shared-volume/_data",
    "Name": "my-shared-volume",
    "Options": null,
    "Scope": "local"
  }
]

❏ [19:52:08]
> docker volume ls
DRIVER      VOLUME NAME
local      88aa686723ad3f1f0c662cd8120c8e45a8cddb77961bcc38364d1ebb8ea8db9
local      minikube
local      my-shared-volume

❏ [19:52:15]
>
```

Part 2: You need to use **bind mounts** to share a directory between two containers

1. 在主機上建立一個目錄

```
mkdir -p ~/shared-data
```

```
cd ~/shared-data
```

```
echo "Initial data from host" > host-file.txt
```

2. 啟動第一個容器，使用 bind mount

```
docker run -dit --name bind-container1 \
```

```
-v ~/shared-data:/data \
```

```
busybox
```

3. 在 bind-container1 中修改檔案

```
docker exec -it bind-container1 sh
```

```
cd /data
```

```
ls -la
cat host-file.txt
echo "Modified by container1" >> host-file.txt
echo "New file from container1" > container1.txt
exit
```

4. 在主機上驗證變更

```
cat ~/shared-data/host-file.txt
ls -la ~/shared-data/
```

5. 啟動第二個容器，掛載同一個主機目錄

```
docker run -dit --name bind-container2 \
-v ~/shared-data:/data \
busybox
```

6. 在 bind-container2 中讀取資料

```
docker exec -it bind-container2 sh
cd /data
ls -la
cat host-file.txt
cat container1.txt
exit
```

7. 清理 (optional)

```
docker stop bind-container1 bind-container2
docker rm bind-container1 bind-container2
```

Step1~4

```

[19:54:40 (43/43)]
> mkdir -p ~/shared-data
[19:55:00]
> cd ~/shared-data
[19:55:15]
> echo "Initial data from host" > host-file.txt
[19:55:33]
> docker run -dit --name bind-container1 \
-v ~/shared-data:/data \
busybox
docker: permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Head "http://v2Fvar%2Frun%2Fdocker.sock/_ping": dial unix /var/run/docker.sock: connect: permission denied
Run 'docker run --help' for more information
[19:55:40]
> docker run -dit --name bind-container2 \
-v ~/shared-data:/data \
busybox
9052b71cd283cc49ac906f81f49231f93bf0911fbc6fa227f140bbb71de6e
[19:56:01]
> docker exec -it bind-container1 sh
/ # cd /data
/data # ls -la
total 12
drwxr-xr-x  2 1045  1045    4096 Nov  5 11:55 .
drwxr-xr-x  1 root   root    4096 Nov  5 11:56 ..
-rw-rw-rw-  1 1045  1045     25 Nov  5 11:55 host-file.txt
/data # cat host-file.txt
Initial data from host
/data # echo "Modified by container1" >> host-file.txt
/data # echo "New file from container1" > container1.txt
/data # exit
[19:57:02]
> cd ~/shared-data/host-file.txt
Initial data from host
Modified by container1
[19:57:07]
> ls -la ~/shared-data/
total 16
drwxr-xr-x  2 DannyTso DannyTso 4096 +-  5 19:56 .
drwxr-xr-x  1 DannyTso DannyTso 4096 +-  5 19:56 ..
-rw-rw-rw-  1 root     root      25 +-  5 19:56 container1.txt
-rw-rw-rw-  1 DannyTso DannyTso 40 +-  5 19:56 host-file.txt
[19:57:13]
> docker run -dit --name bind-container2 \
-v ~/shared-data:/data \
busybox
[19:57:13]

```


Step5~6

```
➤ docker run -dit --name bind-container2 \
-v ~/shared-data:/data \
busybox
d3836d43d1cd9d2a0fb75ef9edfd221887b3d255604519f81a39a551efa5c

➤ docker exec -it bind-container2 sh
/ # cd /data
/data # ls -la
total 16
drwxr-xr-x  2 1045  1045    4096 Nov  5 11:56 .
drwxr-xr-x  1 root   root    4096 Nov  5 11:58 ..
-rw-r--r--  1 root   root     23 Nov  5 11:56 container1.txt
-rw-r--r--  1 1045  1045    4096 Nov  5 11:56 host-file.txt
/data # cat host-file.txt
Initial data from host
Modified by container1
/data # cat container1.txt
Now file from container1
/data # exit
```

2-2: What's the difference between these two steps?

特性	Docker Volume	Bind Mounts
管理方式	Docker 完全管理，資料放在 Docker 專用目錄	直接映射 Host 上的指定目錄或檔案，管理權跟路徑都在主機 OS
可攜性	高：容易搬到其他環境	低：要看主機的檔案結構
同步方式	透過 Docker 管理，container 之間共享	主機跟 container 即時雙向同步
生命週期	Container 刪掉後 volume 還在 (要自己手動刪)	主機目錄不受 container 影響 (container 死了也不影響主機)
使用場景	資料要永久保存 or container 之間共用資料	開發環境 or 需要即時同步的情況
主機 visibility	看不到：你很難直接在主機找到資料在哪 (要用 Docker 指令去看)	看得到：資料就在你指定的主機資料夾，可以直接用記事本或 VSCode 打開來改

簡單講：

- **Volume**：Docker 幫你管，資料獨立存在，適合正式環境跟資料持久化
 - 優點：可攜性好，Docker 統一管理
 - 缺點：不容易直接看到檔案在哪
- **Bind Mounts**：直接把主機目錄掛進 container，即時同步
 - 優點：改了馬上看到效果，適合開發
 - 缺點：綁定主機路徑，不好搬移

Part 3: Answer the following questions

3-1: Please explain the difference between Docker Container and Virtual Machine.

1. 架構上的不同：

VM 跟 Container 最大的不同就是虛擬化的層級不一樣。

- VM 是在硬體層做虛擬化，每個 VM 都要跑整套 OS，透過 Hypervisor 管理。開一台 VM 就等於在你的實體機器上再跑一個完整的作業系統，啟動要等好幾分鐘，而且吃的資源也多。
- Container 則是在 OS 層做隔離，所有 container 共用 Host OS 的 kernel，只是把 app 跟它需要的 libraries 包起來。而且因為不用載入完整 OS，所以啟動超快（幾秒就好），資源占用也少很多。

2. 隔離程度：

- VM：硬體層級隔離，每個 VM 有自己的 kernel，彼此完全分開 ⇒ 安全性高
- Container：用 namespace 和 cgroups 做隔離，但共用同個 kernel ⇒ 隔離程度相對弱

3. 什麼時候用哪個？

- 需要跑不同 OS (Windows + Linux) 或對安全要求很高 ⇒ 用 VM
- 做微服務、CI/CD ⇒ 用 Container (輕量快速，適合快速部署擴展)

3-2: Docker is more unsafe than virtual machine, please explain why and what's causing this issue.

Docker 為了輕量跟效能，用了共享 kernel 的設計，但這也帶來一些安全問題：

1. Kernel 共享的風險

- 所有 container 用同一個 kernel
- kernel 有漏洞 ⇒ 可能影響所有 container 甚至整個 host
- VM 就沒這問題 ⇒ 每個 VM 有自己的 kernel

2. Root 權限問題

- Docker daemon 預設用 root 跑
- container 內如果也是 root ⇒ 萬一被攻破，攻擊者可能「逃出」container 並控制整台 host

3. 隔離層級比較弱

- Container 用 namespace + cgroups 隔離
- 不如 VM 的硬體層級隔離徹底
- 如果有人找到 kernel exploit ⇒ 可能突破隔離

Kubernetes

Task 1: Launching K8s and deploy our containers

Launch your Kubernetes cluster

1. Start the Kubernetes cluster

```
minikube start
```



```

C socket_client.c M ! deployment.yaml M ! service.yaml
k8s > ! deployment.yaml
You, 38 秒前 | 1 author (You)
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: socket-server
5 spec:
6   replicas: 1
7   selector:
8     matchLabels:
9       app: socket-server
10  templates:
11    metadata:
12      labels:
13        app: socket-server
14    spec:
15      containers:
16      - name: socket-server
17        image: server-114062545
18        imagePullPolicy: Never
19        ports:
20        - containerPort: 8888

```

```

C socket_client.c M ! deployment.yaml M ! service.yaml
k8s > ! service.yaml
You, 5 小時前 | 1 author (You)
1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: socket-server-service
5 spec:
6   type: NodePort
7   selector:
8     app: socket-server
9   ports:
10  - port: 8888
11    targetPort: 8888
12    nodePort: 30080

```

kubectl apply -f deployment.yaml
kubectl apply -f service.yaml

```

C ~/virtlab/k8s virtlab-hw2-11 ..... minikube • 21:27:40
> kubectl apply -f deployment.yaml
deployment.apps/socket-server created
service/socket-server-service created

C ~/virtlab/k8s virtlab-hw2-12 ..... minikube • 22:12:09
> kubectl apply -f deployment.yaml
deployment.apps/socket-server created

C ~/virtlab/k8s virtlab-hw2-12 ..... minikube • 22:12:12
> kubectl apply -f service.yaml
service/socket-server-service unchanged

```

2. Check the status of your deployment

kubectl get pods
kubectl get deployment
kubectl get services

```

C ~/virtlab/k8s virtlab-hw2-12 ..... minikube • 22:12:18
> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
socket-server-6d595cbdf-tfpfp       1/1     Running   0           64s

C ~/virtlab/k8s virtlab-hw2-12 ..... minikube • 22:13:16
> kubectl get deployment
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
socket-server  1/1     1             1           69s

C ~/virtlab/k8s virtlab-hw2-12 ..... minikube • 22:13:21
> kubectl get services
NAME            TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes     ClusterIP   10.96.0.1    <none>         443/TCP          52m
socket-server-service NodePort    10.97.41.85  <none>         8888:30080/TCP   42m

```

3. Answer the following questions and include the answers in your report:

1. What is Kubernetes? Why do we need it?

K8s 是容器编排平台，用來自動管理一堆 container 的部署、擴展、維護。

為何需要？

- 幾個 container 還能手動管，但幾十上百個管理起來就很麻煩
- K8s 可以幫忙做到：
 - 自動部署、擴展、container 掛了自動重啟
 - 高可用性：有問題馬上啟動新的

- 負載平衡：流量大時自動分散
- 資源優化：自動調配硬體
- 統一管理所有 containers

2. Please explain the concepts of Deployment, Service and Pod.

- **Pod** 是 Kubernetes 最小的部署單位，可以想成是一個或多個容器的「套房」。同一個 Pod 裡的容器會共享網路和儲存空間，就像室友一樣共用同一個 IP 位址和 Port。
- **Deployment** 負責管理 Pod 的 life cycle，你可以告訴它「我要跑 3 個 replica」，它就會確保一直有 3 個 Pod 在運行。如果有某個 Pod 掛了，它會自動補上新的。更新應用程式時，它甚至還能做到絲滑地滾動式更新，不會讓服務中斷。
- **Service** 就像是 Pod 的「門牌號碼」。因為 Pod 可能會被重啟，IP 會變來變去，但 Service 的 IP 是固定的。這樣其他服務就可以透過 Service 來穩定地訪問你的應用，不用擔心 Pod IP 改變的問題。Service 還有 load-balancing 功能，會把流量分散到多個 Pod 上。

3. Why k8s use pod to manipulate the application instead of the container.

1. 有些應用需要多個容器協同工作

比如一個 Web 應用可能需要主程式容器加上一個日誌收集的 sidecar 容器，這兩個容器需要緊密配合，放在同一個 Pod 裡最為合適。

2. 資源共享更方便

同一個 Pod 內的容器可以直接透過 localhost 互相溝通，不需要走網路。它們還能共享 volume，不用擔心資料同步的問題。

3. 保證一起部署

Pod 內的容器會被調度到同一個 node 上，保證它們能正常協作，不會被分配到不同機器上。

4. 更高層次的抽象化

Pod 把邏輯上屬於同一個應用的容器組合在一起，管理起來更加直觀。你不用去記每個容器的關係，單看 Pod 就知道哪些容器是一組的。

5. 簡化編排邏輯

Kubernetes 只需要管理 Pod，不用去追蹤每個容器的狀態，大幅簡化了系統設計。

4. What is Replica Set in Kubernetes?

Replica Set 確保指定數量的 Pod replica 始終在運行。比如你設定要 3 個 replica，它就會確保一直有 3 個 Pod 運作。Pod 掛了會自動補上新的，需要擴展時也能快速增加 the number of replicas。

但 in practical，通常不會直接操作 Replica Set，而是透過 Deployment 來管理。

5. Why can the client successfully communicate through port 30080 when the server is listening on port 8888? (10 points)

原因：Service 的 Port 映射機制

Kubernetes Service 透過三層 port 映射，將外部請求 route 到容器內部：

```
Service:
  type: NodePort
```

```
ports:
- port: 8888      # Service 內部 port
  targetPort: 8888 # Container 實際監聽的 port
  nodePort: 30080 # 外部訪問 port
```

通訊流程：

1. Client 連接到 `NodeIP:30080`
2. Kubernetes 在節點接收 30080 的請求
3. Service 轉發到內部 `ClusterIP:8888`
4. 路由到 Pod 的 `targetPort:8888`
5. Server Container 在 8888 處理請求

為什麼這樣可以成功通訊？

Service 作為中間層，負責 port 轉換和流量 routing。當 Client 訪問 30080 時，Service 自動將請求映射到 Container 監聽的 8888 port，完成整個通訊過程。

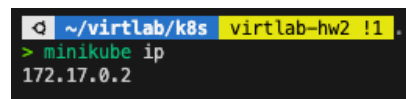
這樣設計的好處：

- 外部 port 跟內部 port 可以不同
- 外部用標準 NodePort 範圍 (30000-32767)
- container 內部可以用任意 port
- 有彈性也比較安全

(Optional) Check if your container is running correctly on the minikube

1. Get the ip address of the minikube:

```
minikube ip
```



```
~/virtlab/k8s virtlab-hw2 !1 ..
> minikube ip
172.17.0.2
```

2. Enter the client folder, open and modify the `PLEASE ASSIGN` section in `socket_client.c`. (Replace with your actual student ID):

修改 socket_client.c 檔案 (Kubernetes 環境)

Steps:

1. 修改 SERVER_ADDRESS

```
#define SERVER_ADDRESS "172.17.0.2" // 實際 IP 需執行 minikube ip 取得
```

原因:

- 在 Kubernetes 環境中, 需要連接到 Minikube 的 IP 位址
- 使用 `minikube ip` 指令取得當前 Minikube 的 IP (例如: 172.17.0.2)

2. 修改 SERVER_PORT

```
#define SERVER_PORT 30080
```

原因:

- 30080 是 Kubernetes Service 的 NodePort
- 此 port 定義在 `service.yaml` 的 `nodePort` 欄位
- 客戶端透過 NodePort 連接到 Kubernetes 叢集內的服務

3. 加入學號到 Hello 訊息

```
char *hello = "Hello from 114062545";
```

原因:

- 同第七節 Docker section 時所提出的作業要求理由

```
C socket_client.c M X / deployment.yaml M / service.yaml
docker > client > C socket_client.c > main()
1 #include <stdio.h>
4 #include <sys/socket.h>
5 #include <arpa/inet.h>
6 #include <unistd.h>
7
8 #define SERVER_ADDRESS "172.17.0.2"
9 #define SERVER_PORT 30080
10
11 int main() {
12     int sock = 0, valread; // you, 3 小時前 • Initial commit
13     struct sockaddr_in serv_addr;
14     char *hello = "Hello from 114062545";
15     char buffer[1024] = {0};
16
17     // 建立 socket
18     if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
19         printf("Socket creation error \n");
20         return -1;
21     }
22
23     // 設置 socket 通訊
24     memset(&serv_addr, '0', sizeof(serv_addr));
25     serv_addr.sin_family = AF_INET;
26     serv_addr.sin_port = htons(SERVER_PORT);
27
28     // 將 IP 位址轉換為網際網路位址
29
30     while (1) {
31         valread = read(sock, buffer, 1024);
32         printf("Received %s\n", buffer);
33         printf("Hello from 114062545\n");
34     }
35 }
```

```
kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
socket-server-6d5895cdf-tpfpj       1/1     Running   0           64s

kubectl get deployment
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
socket-server  1/1     1             1           65s

kubectl get services
NAME                                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
kubernetes  ClusterIP   10.96.0.1     <none>        443/TCP    52m
socket-server-service  NodePort    10.97.41.85   <none>        30080/TCP  42m

minikube ip
172.17.0.2
```

3. Build and run the client container.

Steps:

1. 確保在 Minikube 的 Docker 環境中

```
eval $(minikube docker-env)
```

p.s. 可用 `echo $DOCKER_HOST` 檢查輸出

```
~/virtlab/k8s virtlab-hw2 !1
> echo $DOCKER_HOST
tcp://172.17.0.2:2376
```

2. 進入 client 資料夾

```
cd ..
cd docker/client
```

3. 重新建立 client 映像檔 (只要是更新過 socket_client.c 就需重建)

```
docker build -t client-114062545 .
```

```
~/virtlab docker/client virtlab-hw2 !1
> docker client
docker build -t client-114062545 .
[+] Building 17.5s (9/9) FINISHED
=> [internal] load .dockerignore
=> [internal] load build definition from Dockerfile
=> [internal] load metadata for docker.io/library/gcc:12.2.0
=> [internal] load build context
=> transferring context: 1.38kB
[1/4] FROM docker.io/library/gcc:12.2.0@sha256:a3e091325c8af43bc8c1c76d8d153351d0b16381244211880b4b4fcacc1452
=> resolve docker.io/library/gcc:12.2.0@sha256:a3e091325c8af43bc8c1c76d8d153351d0b16381244211880b4b4fcacc1452
=> sha256:5d790e3a81c561833dc654d4e647d40121a59e1a9a178743e3085455475e 15.76MB / 15.76MB
=> sha256:d01a6a7c087a824580388c6a9dca5113d530bcb2c04e9a4a539904f47c 8.68kB / 8.68kB
=> sha256:918547b9432087b1e0238e82c1e0a0b73aa9f3c402ee9a8c6d01a8cb55 55.89MB / 55.89MB
=> sha256:4e0d9c3ab7e5f63753a5e788cb0872a715f1141e8ce5a87638e6cd370a41 54.59MB / 54.59MB
=> sha256:a3e091325c8af43bc8c1c76d8d153351d0b16381244211880b4b4fcacc1452 1.43kB / 1.43kB
=> sha256:8a340a0a8e147a55c51b791244556ddc17024f0b132606cd08ac67130 2.00kB / 2.00kB
=> sha256:9cda044053f027e7740a38729fb1ee76022e18370ebdc3502e66f79af3f 196.85MB / 196.85MB
=> extracting sha256:918547b9432087b1e0238e82c1e0a0b73aa9f3c402ee9a8c6d01a8cb55
=> sha256:7ef776ce99b1d9f65a978fa5673066e40c14c2da723c8e44ba0d8ce4d0 17.13kB / 17.13kB
=> sha256:6d02efafaff048565c3ebd8623e5d4437f1f68cd7635aa43735e1a7bbd251 138.30MB / 138.30MB
=> sha256:3f2f807db31c52a8e6ea873ab3aed3f0101790e145d576c396ae3411a7c 10.13kB / 10.13kB
=> extracting sha256:5c7040db1c53113365464e47d40121a59e1a9a178743e3085455475e
=> sha256:e06ccbcfc97c154d4721a6c1c56410e41750410570ba1fc3ea5c10d715804 1.88kB / 1.88kB
=> extracting sha256:4e0d9c3ab7e5f63753a5e788cb0872a715f1141e8ce5a87638e6cd370a41
=> extracting sha256:3c0da0a05f0ef740a38729fb1ee76022e18370ebdc3502e66f79af3f
=> extracting sha256:7dcff776ce99b1d9f65a978fa5673066e40c14c2da723c8e44ba0d8ce4d0
=> extracting sha256:6d02efafaff048565c3ebd8623e5d4437f1f68cd7635aa43735e1a7bbd251
=> extracting sha256:a3e091325c8af43bc8c1c76d8d153351d0b16381244211880b4b4fcacc1452
=> [2/4] COPY socket_client.c /src/socket_client.c
=> [3/4] WORKDIR /src
=> [4/4] RUN gcc -o client socket_client.c
=> exporting to image
=> exporting layers
=> writing image sha256:e14115d08b51899af26a8312f507e0b48fb7f9ccd7091ce4730847cafa433c
=> naming to docker.io/library/client-114062545

~/virtlab docker/client virtlab-hw2 !1
> docker images
REPOSITORY          TAG         IMAGE ID      CREATED       SIZE
client-114062545    latest      e14115d08b58 10 seconds ago 1.27GB
registry.k8s.io/kube-apiserver    v1.27.4    e7972205b661 2 years ago   121MB
registry.k8s.io/kube-controller-manager v1.27.4    f46646886407 2 years ago   113MB
registry.k8s.io/kube-proxy        v1.27.4    6846c72e0a34 2 years ago   71.1kB
registry.k8s.io/kube-scheduler    v1.27.4    98ef2578f3cd 2 years ago   58.4MB
registry.k8s.io/coredns/coredns   v1.10.1    ea0b44c33d48 2 years ago   53.6MB
registry.k8s.io/etcd               3.5.7-b    8b6a47f60522 2 years ago   299MB
registry.k8s.io/pause              3.9        e6f181688397 3 years ago   744kB
gcr.io/k8s-minikube/storage-provisioner v5         6e38740d62bd 4 years ago   31.9MB
```

4. 運行 client 容器

```
docker run -it --rm --network host \
--name client \
client-114062545
```



```
~/virtlab/docker/client virtlab-hw2 !2
> docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
server-114062545    latest     315d88472fa7  8 minutes ago  1.27GB
client-114062545    latest     e14115d08b58  29 minutes ago 1.27GB
registry.k8s.io/kube-apiserver    v1.27.4    e7972205b661  2 years ago   121MB
registry.k8s.io/kube-proxy        v1.27.4    6848d7eda034  2 years ago   71.1MB
registry.k8s.io/kube-scheduler    v1.27.4    98ef2570f3cd  2 years ago   58.4MB
registry.k8s.io/kube-controller-manager v1.27.4    f466468864b7  2 years ago   113MB
registry.k8s.io/coredns/coredns   v1.10.1    ead0a4a53df8  2 years ago   53.6MB
registry.k8s.io/etcd               3.5.7-0    86b6af7dd652  2 years ago   296MB
registry.k8s.io/pause              3.9        e6f181688397  3 years ago   744kB
gcr.io/k8s-minikube/storage-provisioner v5         6e38f40d628d  4 years ago   31.5MB

~/virtlab/docker/client virtlab-hw2 !2
> docker run -it --rm --network host \
  --name client \
  client-114062545
Hello message sent.
Hello from Server
```

Tear down the deployed services and minikube cluster after you finished testing:

```
kubectl delete -f service.yaml
kubectl delete -f deployment.yaml
minikube stop
```

```
~/virtlab/k8s virtlab-hw2 !2
> kubectl delete -f service.yaml
service "socket-server-service" deleted

~/virtlab/k8s virtlab-hw2 !2
> kubectl delete -f deployment.yaml
deployment.apps "socket-server" deleted

~/virtlab/k8s virtlab-hw2 !2
> minikube stop
👉 Stopping node "minikube" ...
🔴 Powering off "minikube" via SSH ...
🔴 1 node stopped.
```

Helper: Using K8sGPT to help debugging your issue

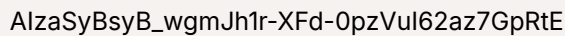
1. Install K8sGPT

```
# 1. 下載最新版本的 .deb 檔案 (v0.4.26)
curl -LO https://github.com/k8sgpt-ai/k8sgpt/releases/download/v0.4.26/k8sgpt_amd64.deb

# 2. 安裝或升級套件
sudo dpkg -i k8sgpt_amd64.deb
```

k8sgpt version

2. Generate API Key from Google AI Studio



```
k8s gpt auth add --backend google --model gemini-2.5-flash --password "AlzaSyBsyB_wgmJh1r-XFd-0pzVu162az7GpRtE"
```

```
k8sgpt analyze --explain --backend google
```

```
問題 輸出 偵錯主控台 終端機 連接埠 GITLENS
15:02:00
> k8sgpt analyze --explain --backend google
100% | (9/9, 12 it/min)
AI Provider: google

0: ConfigMap default/kube-root-ca.crt()
- Error: ConfigMap kube-root-ca.crt is not used by any pods in the namespace
Error: ConfigMap 'kube-root-ca.crt' exists but no pods are using its data.
Solution:
1. Edit pod/deployment YAML.
2. Add 'volumes' to pod spec (type 'configMap', name 'kube-root-ca.crt').
3. Add 'volumeMounts' to container (e.g., 'mountPath: /etc/ssl/certs/ca').
4. Apply changes.

1: ConfigMap kube-node-lease/kube-root-ca.crt()
- Error: ConfigMap kube-root-ca.crt is not used by any pods in the namespace
Error: ConfigMap 'kube-root-ca.crt' exists but no pods are using its data.
Solution:
1. Edit pod/deployment YAML.
2. Add 'volumes' to pod spec (type 'configMap', name 'kube-root-ca.crt').
3. Add 'volumeMounts' to container (e.g., 'mountPath: /etc/ssl/certs/ca').
4. Apply changes.

2: ConfigMap kube-public/cluster-info()
- Error: ConfigMap cluster-info is not used by any pods in the namespace
Error: The 'cluster-info' ConfigMap exists but no pods are configured to use it.
Solution: 1. Verify if 'cluster-info' is still needed. 2. If not, delete it: 'kubectl delete cm cluster-info'.

3: ConfigMap kube-public/kube-root-ca.crt()
- Error: ConfigMap kube-root-ca.crt is not used by any pods in the namespace
Error: ConfigMap 'kube-root-ca.crt' exists but no pods are using its data.
Solution:
1. Edit pod/deployment YAML.
2. Add 'volumes' to pod spec (type 'configMap', name 'kube-root-ca.crt').
3. Add 'volumeMounts' to container (e.g., 'mountPath: /etc/ssl/certs/ca').
4. Apply changes.

4: ConfigMap kube-system/extension-apiserver-authentication()
- Error: ConfigMap extension-apiserver-authentication is not used by any pods in the namespace
Error: This ConfigMap exists but no pods are configured to use it in the namespace. It's an unused resource.
Solution: 1. Confirm it's not needed. 2. If unused, delete it: 'kubectl delete cm extension-apiserver-authentication -n <namespace>'.

5: ConfigMap kube-system/kube-apiserver-legacy-service-account-token-tracking()
- Error: ConfigMap kube-apiserver-legacy-service-account-token-tracking is not used by any pods in the namespace
Error: The ConfigMap 'kube-apiserver-legacy-service-account-token-tracking' exists but no pods are currently using it, likely a leftover or deprecated resource.
Solution: 1. Verify no critical apps depend on it. 2. Delete it: 'kubectl delete cm kube-apiserver-legacy-service-account-token-tracking -n <namespace>'

6: ConfigMap kube-system/kube-root-ca.crt()
- Error: ConfigMap kube-root-ca.crt is not used by any pods in the namespace
Error: ConfigMap 'kube-root-ca.crt' exists but no pods are using its data.
Solution:
1. Edit pod/deployment YAML.
2. Add 'volumes' to pod spec (type 'configMap', name 'kube-root-ca.crt').
3. Add 'volumeMounts' to container (e.g., 'mountPath: /etc/ssl/certs/ca').
4. Apply changes.

7: ConfigMap kube-system/kubeadm-config()
- Error: ConfigMap kubeadm-config is not used by any pods in the namespace
Error: ConfigMap 'kubeadm-config' exists but no pods are currently configured to use it.
Solution: 1. Verify if 'kubeadm-config' is still needed. 2. If not, delete it: 'kubectl delete cm kubeadm-config'.

8: ConfigMap kube-system/kubelet-config()
- Error: ConfigMap kubelet-config is not used by any pods in the namespace
Error: The 'kubelet-config' ConfigMap exists but no pods are currently configured to mount or use it.
Solution: 1. Confirm if this ConfigMap is still required. 2. If needed, update relevant pod/deployment specs to mount it. 3. If not, delete it: 'kubectl delete cm kubelet-config'.
```